>_ MCMP113 · Week 11

Multi-Dimensional Arrays (Matrices)

`int[,]`, nested loops, rows, columns and diagonals, matrix arithmetic, grids and jagged arrays

Asst. Prof. Hasan Oğuz, PhD
Introduction to Basic Algorithms and Programming · Istanbul Okan University
Vocational School · 2026–2027 Fall

How to use these notes

These notes follow the Week 11 slides and add what does not fit on a slide: the exact rules for `int[,]`, drawings of every matrix with its row and column indices, the mathematics behind matrix multiplication, all twelve example programs (`11_01` ... `11_12` in the `code` folder) with real sample runs, the common mistakes, a glossary and graded exercises. Long programs are shown as excerpts; the full, tested file is always in the `code` folder — open it in VS Code and run it while you read. Typed input in the sample runs is shown in **amber**. As in Weeks 9 and 10, everything is written with top-level statements: methods come next week.

Contents

1 Learning outcomes **2**

2 From one index to two **3**

3 Rectangular arrays: `int[,]` **3**

3.1 Declaration, creation and indexing . . . 3
3.2 Initializers . . . 4
3.3 How big is it? . . . 4
3.4 Example 1 – set, measure and print a matrix . . . 4

4 Walking through a matrix **5**

4.1 Row-major and column-major order . . . 5
4.2 Printing aligned columns . . . 5
4.3 When indices go wrong . . . 6

5 Rows and columns **6**

5.1 Example 2 – reading a matrix, row sums and column sums . . . 6
5.2 Example 3 – a grade book (students × exams) . . . 7

6 Square matrices: diagonals, transpose, symmetry **9**

6.1 Diagonals and triangles . . . 9
6.2 Example 4 – diagonal sums and the identity matrix . . . 9

6.3 Example 5 – transpose and symmetric check . . . 10

7 Matrix arithmetic 11

7.1 Addition and scalar multiplication . . . 11
7.2 Matrix multiplication: row times column . . . 13
7.3 Example 7 – three nested loops . . . 13
7.4 Properties: $A \cdot B \neq B \cdot A$, and the identity . . . 14

8 Grids in real programs 14

8.1 Example 8 – the multiplication table as a matrix . . . 14
8.2 Example 9 – cinema seat booking (`char[,]`) . . . 15
8.3 Example 10 – tic-tac-toe . . . 17
8.4 Example 11 – an image is a matrix of brightness values . . . 19
8.5 Example 12 – jagged arrays `int[][]` and Pascal's triangle . . . 20

9 Common mistakes 21

10 Summary 22

11 Glossary (English – Turkish) 23

12 Exercises 23

12.1 Check yourself: predict the output . . . 23
12.2 In-class exercises (pairs) . . . 24
12.3 Homework . . . 24

13 References 24

>_ 1 Learning outcomes

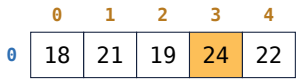
After this week you should be able to:

1. declare and create rectangular arrays (`int[,]`, `double[,]`, `char[,]`), use an initializer, and access an element with `m[r, c]`;
2. use `GetLength(0)` (rows), `GetLength(1)` (columns), `Length` (all elements) and `Rank` correctly;
3. visit every element with two nested `for` loops, in row-major or column-major order, and print a matrix as an aligned grid;
4. compute row sums, column sums, row/column averages and maxima, diagonal sums and the transpose, and check whether a matrix is symmetric;
5. add and scale matrices, and multiply them with $C_{ij} = \sum_k A_{ik} B_{kj}$ after checking the sizes;
6. model real grids (grade books, seat maps, game boards, images) as matrices, converting between "numbers from 1" and indices from 0;
7. use a jagged array `int[][]` when the rows have different lengths.

>_ 2 From one index to two

In Week 9 an array was one row of boxes, and one index was enough: `temps[3]` is the temperature of day 3. Many data sets, however, are *tables*: three weeks of temperatures with five days each, the grades of 40 students in 3 exams, the 5×8 seats of a cinema hall, the pixels of a photo. We could use one 1-D array per row (`week1`, `week2`, `week3`), but that fails for the same reason ten grade variables failed in Week 9: the number of rows is fixed in the program text and no loop can run over the arrays.

A **matrix** (two-dimensional array) keeps the whole table under one name. Every element is found by *two* indices: its **row** and its **column**.



`temps[3]`



`temps[1, 3]`
week 1, day 3

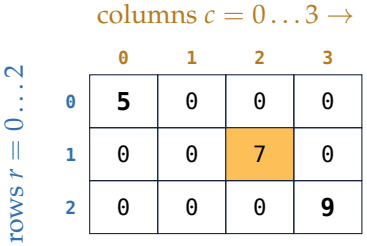
Everything you learned in Week 10 — sum, average, maximum, counting, searching — still works on a matrix. We only add a second, nested loop.

>_ 3 Rectangular arrays: `int[,]`

3.1 Declaration, creation and indexing

```
int[,] m = new int[3, 4];   // 3 rows, 4 columns: 12 elements, all 0
m[1, 2] = 7;                // row 1, column 2
```

- The **comma** inside `[,]` is the whole difference from Week 9: one comma means two dimensions (`int[,,]` would have three).
- In `new int[3, 4]` the **first number is the number of rows**, the second the number of columns. In `m[1, 2]` the first index is the row. Think of a cinema ticket: *row 12, seat 7* — row first.
- Both indices start at 0: valid rows are $0, 1, 2$ and valid columns $0, \ldots, 3$.
- All elements get the **default value** of their type, exactly as in Week 9: 0 for `int`, `long` and `double`, `false` for `bool`, `'\0'` for `char`, `null` for `string`.
- Mathematics writes the element in row i, column j as a_{ij} and counts from 1. The element a_{23} of a maths book is `m[1, 2]` in C#.



`m[0, 0]` = 5 top-left
`m[1, 2]` = 7
`m[2, 3]` = 9 bottom-right

3.2 Initializers

When the values are known in advance we write them row by row; the compiler counts the inner braces (rows) and the values in each (columns):

```
int[,] small = { { 1, 2, 3 }, { 4, 5, 6 } };   // 2 rows, 3 columns; small[1, 0] == 4
```

All rows must have the same length. `| 1, 2 , 3 |` is rejected with `error CS0847: An array initializer of length '2' is expected`.

3.3 How big is it?

Expression	Value for `new int[3, 4]`	Meaning
`m.GetLength(0)`	3	number of rows (dimension 0)
`m.GetLength(1)`	4	number of columns (dimension 1)
`m.Length`	12	number of *all* elements, $3 \cdot 4$
`m.Rank`	2	number of dimensions

Length is not the number of rows

In Week 9 we wrote `|for (int i = 0; i < a.Length; i++)|`. For a matrix, `Length` is the total number of elements (12), so a loop `|r < m.Length|` asks for rows $3, 4, \ldots$ and crashes. Loops over rows use `GetLength(0)`, loops over columns `GetLength(1)`.

3.4 Example 1 – set, measure and print a matrix

Listing 1: 11_01_MatrixBasics/Program.cs

```
// MCMP113 · Week 11 · Example 1 — Matrix basics
// Declares a 2-D array, uses m[row, col] and GetLength, prints it as a grid.

int[,] m = new int[3, 4];        // 3 rows, 4 columns, all 0

m[0, 0] = 5;                     // row 0, column 0 (top-left)
m[1, 2] = 7;                     // row 1, column 2
m[2, 3] = 9;                     // row 2, column 3 (bottom-right)

int rows = m.GetLength(0);       // dimension 0 = rows
int cols = m.GetLength(1);       // dimension 1 = columns
Console.WriteLine($"rows = {rows}, cols = {cols}");
Console.WriteLine($"Length = {m.Length}, Rank = {m.Rank}");

// row-major traversal: outer loop = rows, inner loop = columns
for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < cols; c++)
    {
        Console.Write($"{m[r, c],4}");   // width 4, right-aligned
    }
    Console.WriteLine();                 // end of one row
}

// initializer: the shape comes from the braces (2 rows, 3 columns)
int[,] small = { { 1, 2, 3 }, { 4, 5, 6 } };
Console.WriteLine($"small[1, 0] = {small[1, 0]}");

```

```
// foreach visits every element in row-major order
Console.Write("foreach:");
foreach (int x in small)
{
    Console.Write($" {x}");
}
Console.WriteLine();
```

Sample run

```
rows = 3, cols = 4
Length = 12, Rank = 2
   5   0   0   0
   0   0   7   0
   0   0   0   9
small[1, 0] = 4
foreach: 1 2 3 4 5 6
```

The inner loop prints *one row* with `Console.Write` (no line break); the `WriteLine` after the inner loop ends that row. `foreach` visits all elements in row-major order, but it gives no indices, so for most matrix work we use `for` loops.

>_ 4 Walking through a matrix

4.1 Row-major and column-major order

Two nested loops visit every element exactly once. The **outer loop chooses the line** (a row or a column), the **inner loop walks along it**. The numbers in the cells below are the *order* of the visits, not the values.

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12

row-major: outer loop over r

	0	1	2	3
0	1	4	7	10
1	2	5	8	11
2	3	6	9	12

column-major: outer loop over c

Row-major

FOR $r \leftarrow 0$ **TO** rows -1
  FOR $c \leftarrow 0$ **TO** cols -1
    PRINT $m[r, c]$
  END FOR
END FOR

Column-major

FOR $c \leftarrow 0$ **TO** cols -1
  FOR $r \leftarrow 0$ **TO** rows -1
    PRINT $m[r, c]$
  END FOR
END FOR

In both versions the element is written `m[r, c]` — row first. Only the *order of the loops* changes. C# stores a rectangular array row by row, which is why row-major order is the natural one.

4.2 Printing aligned columns

The matrix | 5, 120, 7 , 40, 3, 1000 | printed with three different inner statements (real runs):

Inner statement	Output
`Console.Write(m[r, c] + " ");`	`5 120 7` `40 3 1000`
`Console.Write($"{m[r, c],5}");`	`    5  120    7` `   40    3 1000`
`Console.Write($"{m[r, c],-5}|");`	`5    |120  |7    |` `40   |3    |1000 |`

`{x,5}` pads the value to at least 5 characters on the left (right-aligned, the usual choice for numbers); `{x,-5}` pads on the right (left-aligned, good for names). Choose the width as the widest value plus one or two spaces.

4.3 When indices go wrong

- **Out of range** (run-time error): for a 3×4 matrix `|m[3, 0]|` compiles but crashes with `Unhandled exception. System.IndexOutOfRangeException: Index was outside the bounds of the array.`
- **Wrong brackets** (compile error): `|m[1][2]|` is the syntax of jagged arrays (Section 8.5). For `int[,]` the compiler reports `error CS0022: Wrong number of indices inside []; expected 2`.
- **Swapped sizes** (silent logic error): `|for (int c = 0; c < m.GetLength(0); c++)|`. On a square matrix the bug is invisible; on a 3×4 matrix the last column is never visited. Always test with a non-square matrix.

>_ 5 Rows and columns

5.1 Example 2 – reading a matrix, row sums and column sums

The sizes are read first; `new int[rows, cols]` accepts variables. The nested loops then ask for every element in row-major order. For a matrix a with R rows and K columns the row sums, column sums and the total are

$$\text{row}_r = \sum_{c=0}^{K-1} a_{rc}, \qquad \text{col}_c = \sum_{r=0}^{R-1} a_{rc}, \qquad \text{total} = \sum_r \text{row}_r = \sum_c \text{col}_c.$$

The last equality is a free test of the program: both ways of adding must give the same total.

Listing 2: 11_02_RowColumnSums/Program.cs

```
1  // MCMP113 · Week 11 · Example 2 — Read a matrix, row sums and column sums
2  // Reads rows x cols numbers from the keyboard, prints the grid with row sums, column sums and the
       total.
3
4  Console.Write("Rows: ");
5  int rows = int.Parse(Console.ReadLine());
6  Console.Write("Columns: ");
7  int cols = int.Parse(Console.ReadLine());
8
9  int[,] a = new int[rows, cols];
10 for (int r = 0; r < rows; r++)
11 {
12     for (int c = 0; c < cols; c++)
13     {
```

```
        Console.Write($"a[{r},{c}] = ");
        a[r, c] = int.Parse(Console.ReadLine());
    }
}

// each row, followed by its sum
int total = 0;
for (int r = 0; r < rows; r++)
{
    int rowSum = 0;          // reset for every row
    for (int c = 0; c < cols; c++)
    {
        Console.Write($"{a[r, c],5}");
        rowSum += a[r, c];
    }
    Console.WriteLine($"  | {rowSum,4}");
    total += rowSum;
}

// column sums: the OUTER loop runs over the columns
for (int c = 0; c < cols; c++)
{
    int colSum = 0;          // reset for every column
    for (int r = 0; r < rows; r++)
    {
        colSum += a[r, c];
    }
    Console.Write($"{colSum,5}");
}
Console.WriteLine($"  | {total,4}");
```

Sample run

```
Rows: 2
Columns: 3
a[0,0] = 4
a[0,1] = 7
a[0,2] = 1
a[1,0] = 2
a[1,1] = 5
a[1,2] = 8
    4    7    1  |   12
    2    5    8  |   15
    6   12    9  |   27
```

	0	1	2	Σ
0	4	7	1	12
1	2	5	8	15
Σ	6	12	9	27

green: row sums (outer loop over rows)
blue: column sums (outer loop over columns)
$12 + 15 = 27 = 6 + 12 + 9$

c (outer)	r (inner)	`colSum += a[r, c]`	printed
0	0, 1	$0 + 4 + 2$	6
1	0, 1	$0 + 7 + 5$	12
2	0, 1	$0 + 1 + 8$	9

The accumulators `rowSum` and `colSum` are declared *inside* the outer loop, so they start at 0 for every row (column). Declared before the outer loop, they would keep growing: for `|  1, 2 ,  3, 4  |` the program would print `Row 0: 3 Row 1: 10` instead of 3 and 7 (real run of the buggy version).

5.2 Example 3 – a grade book (students × exams)

Two 1-D arrays from Week 9 give names to the rows (`students`) and the columns (`exams`) of a `double[,]` matrix. Every row answers a question about a *student* (average, best grade), every

column a question about an *exam* (exam average). The names and grades are invented.

Listing 3: 11_03_GradeBook/Program.cs (lines 1–15 declare `students`, `exams` and the 4×3 matrix `grades` shown in the run below)

```
// one line per student: grades, row average, row maximum
int topStudent = 0;
double topAverage = -1;
for (int r = 0; r < nStudents; r++)
{
    double rowSum = 0;
    double rowMax = grades[r, 0];    // first grade of the row
    Console.Write($"{students[r],-8}");
    for (int c = 0; c < nExams; c++)
    {
        Console.Write($"{grades[r, c],9:F0}");
        rowSum += grades[r, c];
        if (grades[r, c] > rowMax)
        {
            rowMax = grades[r, c];
        }
    }
    double average = rowSum / nExams;
    Console.WriteLine($"{average,9:F2}{rowMax,6:F0}");
    if (average > topAverage)
    {
        topAverage = average;
        topStudent = r;
    }
}

// column averages: outer loop over the exams
Console.Write($"{"Average",-8}");
for (int c = 0; c < nExams; c++)
{
    double colSum = 0;
    for (int r = 0; r < nStudents; r++)
    {
        colSum += grades[r, c];
    }
    Console.Write($"{colSum / nStudents,9:F2}");
}
Console.WriteLine();
Console.Write($"Top student: {students[topStudent]}");
Console.WriteLine($" ({topAverage:F2})");
```

Sample run

```
Student      Quiz  Midterm    Final  Average  Best
Elif           70       65       80    71.67    80
Mehmet         45       55       60    53.33    60
Zeynep         90       85       95    90.00    95
Can            60       75       50    61.67    75
Average     66.25    70.00    71.25
Top student: Zeynep (90.00)
```

Check two numbers by hand: Elif $(70 + 65 + 80)/3 = 215/3 = 71.67$; quiz average $(70 + 45 + 90 + 60)/4 = 265/4 = 66.25$. The row maximum starts with the *first* element of the row, `grades[r, 0]`, never with 0 (Week 10). The search for the top student is the same "running maximum" one level higher: it remembers the *index* `r` of the best row, so that the name can be printed.

>_ 6 Square matrices: diagonals, transpose, symmetry

6.1 Diagonals and triangles

An $n \times n$ matrix has a **main diagonal** ($r = c$, top-left to bottom-right) and an **anti-diagonal** ($r + c = n - 1$, top-right to bottom-left). Above the main diagonal lies the **upper triangle** ($c > r$), below it the **lower triangle** ($c < r$). The sum of the main diagonal is called the **trace**: $\mathrm{tr}(A) = \sum_i a_{ii}$.

	0	1	2	3
0	2	7	1	8
1	7	3	5	0
2	1	5	4	6
3	8	0	6	9

amber: main $r = c$
blue: anti $r + c = 3$

	0	1	2	3
0	2	7	1	8
1	7	3	5	0
2	1	5	4	6
3	8	0	6	9

green: upper $c > r$
red: lower $c < r$

Because one index follows from the other, **one loop is enough** for a diagonal: `a[i, i]` and `a[i, n - 1 - i]`. A nested loop with |if (r == c)| gives the same result with n^2 instead of n steps.

6.2 Example 4 – diagonal sums and the identity matrix

Listing 4: 11_04_Diagonals/Program.cs (lines 1–33 declare the 4×4 matrix `a` of Section 6.1 and print it with `[x]` on the main diagonal and `(x)` on the anti-diagonal, using |if (r == c)| ... |else if (r + c == n - 1)| inside the nested loops)

```
34
35  // one loop is enough: on each diagonal the column follows from the row
36  int mainSum = 0;
37  int antiSum = 0;
38  for (int i = 0; i < n; i++)
39  {
40      mainSum += a[i, i];
41      antiSum += a[i, n - 1 - i];
42  }
43  Console.WriteLine($"Main diagonal sum (trace) = {mainSum}");
44  Console.WriteLine($"Anti-diagonal sum         = {antiSum}");
45
46  // identity matrix of the same size
47  int[,] identity = new int[n, n];     // all zeros
48  for (int i = 0; i < n; i++)
49  {
50      identity[i, i] = 1;
51  }
52  Console.WriteLine($"Identity {n} x {n}:");
53  for (int r = 0; r < n; r++)
54  {
55      for (int c = 0; c < n; c++)
56      {
57          Console.Write($"{identity[r, c],3}");
58      }
59      Console.WriteLine();
60  }
```

Sample run

```
  [2]    7    1  (8)
    7  [3]  (5)    0
    1  (5)  [4]    6
  (8)    0    6  [9]
Main diagonal sum (trace) = 18
Anti-diagonal sum         = 26
Identity 4 x 4:
  1  0  0  0
  0  1  0  0
  0  0  1  0
  0  0  0  1
```

By hand: trace $= 2+3+4+9 = 18$, anti-diagonal $= 8+5+5+8 = 26$.

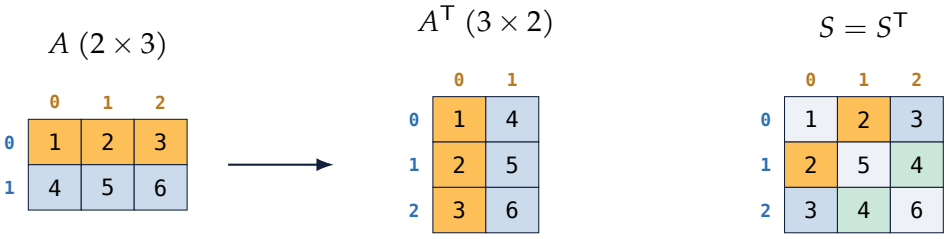
The **identity matrix** I has 1 on the main diagonal and 0 elsewhere:

$$I_{ij} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}$$

`new int[n, n]` is already all zeros, so one loop that sets `identity[i, i] = 1` is enough. For odd n the centre element lies on both diagonals.

6.3 Example 5 – transpose and symmetric check

The **transpose** A^{T} swaps rows and columns: $(A^{\mathsf{T}})_{cr} = a_{rc}$. Row r of A becomes column r of A^{T}, and the size changes from $R \times K$ to $K \times R$. Therefore the new array must be created with the *sizes swapped*: `new int[cols, rows]`. With `new int[rows, cols]` a 2×3 matrix crashes at `t[2, 0]`.



A square matrix S is **symmetric** if $S^{\mathsf{T}} = S$, i.e. $s_{rc} = s_{cr}$ for all r, c (same colour = mirror pair). We do not have to build the transpose: we compare each element of the upper triangle with its mirror image. Starting the inner loop at `c = r + 1` skips the diagonal and the pairs already compared, so only $\frac{n(n-1)}{2}$ comparisons are made (3 for $n = 3$, instead of 9).

Listing 5: 11_05_TransposeSymmetric/Program.cs (the printing loops, lines 21–38, are omitted)

```
1  // MCMP113 · Week 11 · Example 5 — Transpose and symmetric check
2  // The transpose swaps rows and columns: t[c, r] = a[r, c]. A square matrix is symmetric if s[r, c]
       == s[c, r].
3
4  int[,] a =
5  {
6      { 1, 2, 3 },
7      { 4, 5, 6 },
8  };
9  int rows = a.GetLength(0);           // 2
10 int cols = a.GetLength(1);           // 3
11
```

```
int[,] t = new int[cols, rows];      // 3 x 2: the sizes are swapped
for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < cols; c++)
    {
        t[c, r] = a[r, c];           // row r of a becomes column r of t
    }
}

// symmetric check: compare each element above the diagonal
// with its mirror image below the diagonal
int[,] s =
{
    { 1, 2, 3 },
    { 2, 5, 4 },
    { 3, 4, 6 },
};
int n = s.GetLength(0);
bool symmetric = true;
int checks = 0;
for (int r = 0; r < n; r++)
{
    for (int c = r + 1; c < n; c++)  // start right of the diagonal
    {
        checks++;
        if (s[r, c] != s[c, r])
        {
            symmetric = false;
        }
    }
}
Console.WriteLine($"S symmetric? {symmetric}  ({checks} comparisons)");
```

Sample run

```
A (2 x 3):
   1   2   3
   4   5   6
Transpose (3 x 2):
   1   4
   2   5
   3   6
S symmetric? True  (3 comparisons)
```

The pattern "assume `true`, look for one counterexample" is the same as in the prime test of Week 6.

>_ 7 Matrix arithmetic

7.1 Addition and scalar multiplication

For two matrices of the *same size* $R \times K$, and a number k,

$$(A + B)_{rc} = a_{rc} + b_{rc}, \qquad (k \cdot A)_{rc} = k \cdot a_{rc}.$$

Addition is element by element and is not defined for different sizes, so the program checks both `GetLength` values first. Scalar multiplication is always defined.

Listing 6: 11_06_AddScale/Program.cs (from line 13; the matrices are $A = \left(\begin{smallmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{smallmatrix}\right)$, $B = \left(\begin{smallmatrix} 10 & 20 & 30 \\ -1 & -2 & -3 \end{smallmatrix}\right)$)

```
};
Console.Write("Scalar k = ");
int k = int.Parse(Console.ReadLine());

int rows = a.GetLength(0);
int cols = a.GetLength(1);

// addition: sizes must match exactly
if (rows != b.GetLength(0) || cols != b.GetLength(1))
{
    Console.WriteLine("A + B is not defined: different sizes.");
}
else
{
    int[,] sum = new int[rows, cols];
    for (int r = 0; r < rows; r++)
    {
        for (int c = 0; c < cols; c++)
        {
            sum[r, c] = a[r, c] + b[r, c];   // element by element
        }
    }
    Console.WriteLine("A + B =");
    for (int r = 0; r < rows; r++)
    {
        for (int c = 0; c < cols; c++)
        {
            Console.Write($"{sum[r, c],5}");
        }
        Console.WriteLine();
    }
}

// scalar multiplication: always defined
int[,] scaled = new int[rows, cols];
for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < cols; c++)
    {
        scaled[r, c] = k * a[r, c];
    }
}
Console.WriteLine($"{k} * A =");
for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < cols; c++)
    {
        Console.Write($"{scaled[r, c],5}");
    }
    Console.WriteLine();
}
```

Sample run

```
Scalar k = 3
A + B =
   11   22   33
    3    3    3
3 * A =
    3    6    9
   12   15   18
```

7.2 Matrix multiplication: row times column

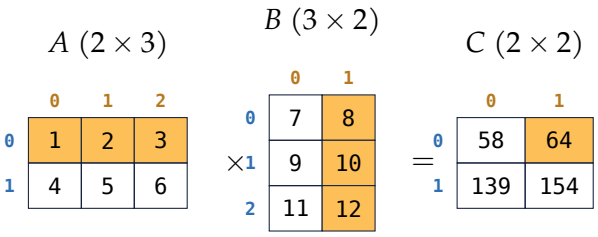
Matrix multiplication is *not* element by element. If A is $m \times n$ and B is $n \times p$, the product $C = A \cdot B$ is the $m \times p$ matrix with

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik}\, B_{kj} \qquad (0 \le i < m,\ 0 \le j < p).$$

To get one element C_{ij}, take **row** i **of** A and **column** j **of** B, multiply them pair by pair and add. The pairs only match if the row of A is as long as the column of B:

$$(m \times \underline{n}) \cdot (\underline{n} \times p) \longrightarrow m \times p.$$

The inner sizes must be equal; the outer sizes give the size of C. Otherwise $A \cdot B$ is not defined.



$C_{01} = 1 \cdot 8 + 2 \cdot 10 + 3 \cdot 12 = 8 + 20 + 36 = 64, \qquad C_{10} = 4 \cdot 7 + 5 \cdot 9 + 6 \cdot 11 = 28 + 45 + 66 = 139.$

In the program, i runs over the rows of C, j over its columns and k along row i of A and column j of B. The trace of the innermost loop for C_{11}:

k	A_{1k}	B_{k1}	product	`c[1, 1]`
—				0
0	4	8	32	32
1	5	10	50	82
2	6	12	72	**154**

7.3 Example 7 – three nested loops

Listing 7: 11_07_MatrixMultiply/Program.cs (lines 1–14 declare the 2×3 matrix `a` and the 3×2 matrix `b` shown above)

```
16 int m = a.GetLength(0);
17 int n = a.GetLength(1);
18 int p = b.GetLength(1);
19
20 if (n != b.GetLength(0))         // columns of A must = rows of B
21 {
22     Console.WriteLine("A x B is not defined.");
23 }
24 else
25 {
26     int[,] c = new int[m, p];        // result: m x p, starts at 0
27     for (int i = 0; i < m; i++)
28     {
29         for (int j = 0; j < p; j++)
30         {
31             for (int k = 0; k < n; k++)
32             {
```

```
                // row i of A times column j of B
                c[i, j] += a[i, k] * b[k, j];
            }
        }
    }
    Console.WriteLine($"A ({m}x{n}) x B ({n}x{p}) = C ({m}x{p}):");
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < p; j++)
        {
            Console.Write($"{c[i, j],6}");
        }
        Console.WriteLine();
    }
    Console.WriteLine($"Multiplications: {m * p * n}");
}
```

Sample run

```
A (2x3) x B (3x2) = C (2x2):
    58    64
   139   154
Multiplications: 12
```

`c[i, j]` can be accumulated with `+=` directly because `new int[m, p]` starts with zeros. The program performs $m \cdot p \cdot n = 2 \cdot 2 \cdot 3 = 12$ multiplications. For two $n \times n$ matrices that is n^3: 1 000 for $n = 10$, 10^6 for $n = 100$, 10^9 for $n = 1000$ — the triple loop is an $O(n^3)$ algorithm (compare bubble sort, $O(n^2)$, Week 10).

7.4 Properties: $A \cdot B \neq B \cdot A$, and the identity

Matrix multiplication is **not commutative**. Running Example 7 with the initializers replaced by 2×2 matrices gives (real runs):

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix}, \qquad \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} = \begin{pmatrix} 3 & 4 \\ 1 & 2 \end{pmatrix}.$$

Multiplying by this "swap" matrix from the right swaps the *columns* of A, from the left it swaps the *rows*. Often $B \cdot A$ is not even defined, or has another size: for the 2×3 and 3×2 matrices above, $A \cdot B$ is 2×2 but $B \cdot A$ is 3×3. The identity matrix plays the role of the number 1: $A \cdot I = I \cdot A = A$ for every square A of the same size.

>_ 8 Grids in real programs

8.1 Example 8 – the multiplication table as a matrix

In Week 6 nested loops *printed* the multiplication table. Now we *store* it first, so that values can be looked up later. Index 0 stands for the number 1, hence `table[r, c]` $= (r+1)(c+1)$ and 7×8 is stored at `table[6, 7]`.

Listing 8: 11_08_MultiplicationTable/Program.cs

```
// MCMP113 · Week 11 · Example 8 – Multiplication table stored in a matrix
// table[r, c] holds (r + 1) * (c + 1): index 0 stands for the number 1.

Console.Write("Size n (1-12): ");
```

```
int n = int.Parse(Console.ReadLine());

int[,] table = new int[n, n];
for (int r = 0; r < n; r++)
{
    for (int c = 0; c < n; c++)
    {
        table[r, c] = (r + 1) * (c + 1);
    }
}

// header row
Console.Write("   x|");
for (int c = 0; c < n; c++)
{
    Console.Write($"{c + 1,4}");
}
Console.WriteLine();

// one line per row, with the row number in front
for (int r = 0; r < n; r++)
{
    Console.Write($"{r + 1,4}|");
    for (int c = 0; c < n; c++)
    {
        Console.Write($"{table[r, c],4}");
    }
    Console.WriteLine();
}

// look-up: 7 x 8 is stored at [6, 7] (if the table is big enough)
if (n >= 8)
{
    Console.WriteLine($"7 x 8 = table[6, 7] = {table[6, 7]}");
}
```

Sample run

```
Size n (1-12): 9
   x|   1   2   3   4   5   6   7   8   9
   1|   1   2   3   4   5   6   7   8   9
   2|   2   4   6   8  10  12  14  16  18
   3|   3   6   9  12  15  18  21  24  27
   4|   4   8  12  16  20  24  28  32  36
   5|   5  10  15  20  25  30  35  40  45
   6|   6  12  18  24  30  36  42  48  54
   7|   7  14  21  28  35  42  49  56  63
   8|   8  16  24  32  40  48  56  64  72
   9|   9  18  27  36  45  54  63  72  81
7 x 8 = table[6, 7] = 56
```

For $n = 5$ the look-up line is skipped, because `table[6, 7]` would not exist: the guard `if (n >= 8)` prevents an `IndexOutOfRangeException`. An alternative is `new int[n + 1, n + 1]` with row 0 and column 0 unused, so that `table[7, 8]` is 56.

8.2 Example 9 – cinema seat booking (`char[,]`)

Every element is a `char`: `'.'` free, `'X'` sold. People count rows and seats from 1, the array from 0; the program converts *once*, with `row - 1` and `seat - 1`, exactly where the array is used. The range check must come first: with row 6, `seats[5, 0]` would crash, and the short-circuit `||` (Week 4)

stops at the first true condition.

Listing 9: 11_09_CinemaBooking/Program.cs (lines 1–17 create the 5 × 8 hall, fill it with `'.'` and mark row 3, seats 4 and 5 as sold)

```
int row;
do
{
    Console.Write($"Row (1-{rows}, 0 = finish): ");
    row = int.Parse(Console.ReadLine());
    if (row != 0)
    {
        Console.Write($"Seat (1-{perRow}): ");
        int seat = int.Parse(Console.ReadLine());
        if (row < 1 || row > rows || seat < 1 || seat > perRow)
        {
            Console.WriteLine("  No such seat.");
        }
        else if (seats[row - 1, seat - 1] == 'X')   // number - 1 = index
        {
            Console.WriteLine("  Sorry, already sold.");
        }
        else
        {
            seats[row - 1, seat - 1] = 'X';
            Console.WriteLine($"  Booked: row {row}, seat {seat}.");
        }
    }
} while (row != 0);

// seat map with the screen at the top, and a count of free seats
Console.WriteLine("       S C R E E N");
Console.Write("    ");
for (int c = 0; c < perRow; c++)
{
    Console.Write($"{c + 1,2}");
}
Console.WriteLine();
int free = 0;
for (int r = 0; r < rows; r++)
{
    Console.Write($"Row{r + 1}");
    for (int c = 0; c < perRow; c++)
    {
        Console.Write($" {seats[r, c]}");
        if (seats[r, c] == '.')
        {
            free++;
        }
    }
    Console.WriteLine();
}
Console.WriteLine($"Free seats: {free} of {seats.Length}");
```

Sample run (part 1)

```
Cinema: 5 rows x 8 seats
Row (1-5, 0 = finish): 3
Seat (1-8): 4
  Sorry, already sold.
Row (1-5, 0 = finish): 3
Seat (1-8): 6
  Booked: row 3, seat 6.
Row (1-5, 0 = finish): 3
Seat (1-8): 4
  Sorry, already sold.
Row (1-5, 0 = finish): 6
Seat (1-8): 1
  No such seat.
Row (1-5, 0 = finish): 1
Seat (1-8): 8
  Booked: row 1, seat 8.
Row (1-5, 0 = finish): 0
```

(part 2)

```
       S C R E E N
     1 2 3 4 5 6 7 8
Row1 . . . . . . . X
Row2 . . . . . . . .
Row3 . . . X X X . .
Row4 . . . . . . . .
Row5 . . . . . . . .
Free seats: 36 of 40
```

$40 - 2$ sold before -2 booked $= 36$. Here `seats.Length` (40) is used correctly: it is the *total* number of seats.

8.3 Example 10 – tic-tac-toe

The board is a 3×3 `char` matrix. There are exactly 8 winning lines: 3 rows, 3 columns and the 2 diagonals of Section 6 ($r = c$ and $r + c = 2$). Only the player who has just moved can have won, so every test compares with `player`.

Listing 10: 11_10_TicTacToe/Program.cs (the board is created and filled with `'.'` in lines 1–11)

```
13 char player = 'X';
14 int moves = 0;
15 bool win = false;
16 while (!win && moves < 9)
17 {
18     Console.Write($"{player} row: ");
19     int row = int.Parse(Console.ReadLine()) - 1;   // 1-3 -> 0-2
20     Console.Write($"{player} col: ");
21     int col = int.Parse(Console.ReadLine()) - 1;
22     bool inside = row >= 0 && row <= 2 && col >= 0 && col <= 2;
23     if (!inside || board[row, col] != '.')
24     {
25         Console.WriteLine("Invalid move, try again.");
26         continue;                                  // same player again
27     }
28     board[row, col] = player;
29     moves++;
30
31     // print the board
32     for (int r = 0; r < 3; r++)
33     {
34         Console.Write($"  {board[r, 0]} {board[r, 1]}");
35         Console.WriteLine($" {board[r, 2]}");
36     }
37
38     // win check: 3 rows, 3 columns, 2 diagonals
39     for (int i = 0; i < 3; i++)
40     {
41         if (board[i, 0] == player
42             && board[i, 1] == player
43             && board[i, 2] == player)
```

```
        {
            win = true;                                 // row i
        }
        if (board[0, i] == player
            && board[1, i] == player
            && board[2, i] == player)
        {
            win = true;                                 // column i
        }
    }
    if (board[0, 0] == player
        && board[1, 1] == player
        && board[2, 2] == player)
    {
        win = true;                                     // main diagonal
    }
    if (board[0, 2] == player
        && board[1, 1] == player
        && board[2, 0] == player)
    {
        win = true;                                     // anti-diagonal
    }

    if (!win)
    {
        player = player == 'X' ? 'O' : 'X';             // next player
    }
}

if (win)
{
    Console.WriteLine($"{player} wins after {moves} moves!");
}
else
{
    Console.WriteLine("Draw: the board is full.");
}
```

An invalid move (outside the board or on a taken cell) prints a message, and `continue` jumps to the next test of the `while` condition: the same player tries again and `moves` does not grow. The player changes only if nobody has won, so the final message names the winner.

Win (moves 1–3)

```
X row: 2
X col: 2
  . . .
  . X .
  . . .
O row: 1
O col: 1
  O . .
  . X .
  . . .
X row: 1
X col: 3
  O . X
  . X .
  . . .
```

Win (moves 4–5)

```
O row: 2
O col: 2
Invalid move, try again.
O row: 2
O col: 1
  O . X
  O X .
  . . .
X row: 3
X col: 1
  O . X
  O X .
  X . .
X wins after 5 moves!
```

Draw (end of a 2nd run)

```
X row: 3
X col: 3
  X X O
  O O X
  X O X
Draw: the board is full.
```

Moves of the draw: $(1,1)$, $(2,2)$, $(1,2)$, $(1,3)$, $(3,1)$, $(2,1)$, $(2,3)$, $(3,2)$, $(3,3)$. Always test both endings.

8.4 Example 11 – an image is a matrix of brightness values

A grey-scale photo is a matrix with one number per pixel: 0 (black) to 255 (white). We use $0 \ldots 9$ so that the numbers fit on a page; every value v is drawn with the character `shades[v]` of the string `␣.:-=+*#%@` (string indexing, Week 9), twice, because console characters are about twice as tall as wide. Inverting (a "negative") replaces every pixel by $9 - v$ (with real photos $255 - v$); other filters are the same loops with another formula.

Listing 11: 11_11_ImageInvert/Program.cs (lines 44–53 print the inverted image with the same loops as lines 22–32)

```
1  // MCMP113 · Week 11 · Example 11 — A grey-scale image as a matrix of brightness values
2  // Brightness 0 (black) .. 9 (white); inverting means new = 9 - old. Real photos use 0 .. 255 the
       same way.
3
4  int[,] image =
5  {
6      { 0, 6, 9, 6, 0, 6, 9, 6, 0 },
7      { 6, 9, 9, 9, 6, 9, 9, 9, 6 },
8      { 9, 9, 9, 9, 9, 9, 9, 9, 9 },
9      { 3, 9, 9, 9, 9, 9, 9, 9, 3 },
10     { 0, 3, 9, 9, 9, 9, 9, 3, 0 },
11     { 0, 0, 3, 9, 9, 9, 3, 0, 0 },
12     { 0, 0, 0, 3, 9, 3, 0, 0, 0 },
13 };
14 string shades = " .:-=+*#%@";            // shades[v] draws brightness v
15 int rows = image.GetLength(0);
16 int cols = image.GetLength(1);
17
18 // draw the image (each pixel twice, so it is not too narrow)
19 // and add up the brightness
20 int sum = 0;
21 Console.WriteLine("Original:");
22 for (int r = 0; r < rows; r++)
23 {
24     for (int c = 0; c < cols; c++)
25     {
26         char ch = shades[image[r, c]];
27         Console.Write($"{ch}{ch}");
```

```
        sum += image[r, c];
    }
    Console.WriteLine();
}
double average = (double)sum / image.Length;
Console.WriteLine($"Average brightness: {average:F2}");

// invert every pixel in place
for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < cols; c++)
    {
        image[r, c] = 9 - image[r, c];
    }
}
```

Sample run (part 1)

```
Original:
  **@@**  **@@**
**@@@@@@**@@@@@@**
@@@@@@@@@@@@@@@@@@@@
--@@@@@@@@@@@@@@--
  --@@@@@@@@@@--
    --@@@@@@--
      --@@--
Average brightness: 5.76
```

(part 2)

```
Inverted:
@@--  --@@--  --@@
--      --      --

**              **
@@**          **@@
@@@@**      **@@@@
@@@@@@**  **@@@@@@
```

Average brightness: the sum of all 63 values is 363, and $363/63 = 5.76$. Note the cast |(double)sum / image.Length| — without it, integer division would give 5 (Week 4).

8.5 Example 12 – jagged arrays `int[][]` and Pascal's triangle

A **jagged array** is an *array of arrays*: `pascal[r]` is a complete 1-D array (row r) with its own `Length`, and `pascal[r][c]` is one element — two pairs of brackets. |new int[n][]| creates only the n "row slots"; each row is still `null` and must be created with |new int[r + 1]| before it is used (otherwise `NullReferenceException`). In Pascal's triangle every inner number is the sum of the two numbers above it:

$$P_{r,0} = P_{r,r} = 1, \qquad P_{r,c} = P_{r-1,c-1} + P_{r-1,c} \quad (0 < c < r),$$

and $P_{r,c} = \binom{r}{c}$ is the binomial coefficient.

	0	1	2	3	4	5
0	1					
1	1	1				
2	1	2	1			
3	1	3	3	1		
4	1	4	6	4	1	
5	1	5	10	10	5	1

`pascal[5][2] = 4 + 6 = 10`

	int[,]	int[][]
element	`m[r, c]`	`j[r][c]`
shape	`rectangle`	`any row lengths`
rows	`m.GetLength(0)`	`j.Length`
length of row r	`m.GetLength(1)`	`j[r].Length`

Listing 12: 11_12_PascalJagged/Program.cs

```
// MCMP113 · Week 11 · Example 12 — Pascal's triangle in a jagged array int[][]
// Row r has r + 1 elements: each row is its own 1-D array, so the rows have different lengths.

Console.Write("Number of rows: ");
```

```
int n = int.Parse(Console.ReadLine());

int[][] pascal = new int[n][];           // n rows, none created yet
for (int r = 0; r < n; r++)
{
    pascal[r] = new int[r + 1];          // row r gets r + 1 elements
    pascal[r][0] = 1;                    // first and last element are 1
    pascal[r][r] = 1;
    for (int c = 1; c < r; c++)
    {
        // sum of the two numbers above
        pascal[r][c] = pascal[r - 1][c - 1] + pascal[r - 1][c];
    }
}

// print as a triangle: 2 spaces of indent per missing row
for (int r = 0; r < n; r++)
{
    for (int s = 0; s < n - 1 - r; s++)
    {
        Console.Write("  ");
    }
    for (int c = 0; c < pascal[r].Length; c++)
    {
        Console.Write($"{pascal[r][c],4}");
    }
    Console.WriteLine();
}
Console.WriteLine($"Rows: {pascal.Length}");
Console.WriteLine($"Last row length: {pascal[n - 1].Length}");
```

Sample run

```
Number of rows: 6
             1
           1   1
         1   2   1
       1   3   3   1
     1   4   6   4   1
   1   5  10  10   5   1
Rows: 6
Last row length: 6
```

Use a jagged array when the rows really have different lengths (triangles, the lines of a text file, students with different numbers of grades); use `int[,]` for every true rectangle.

>_ 9 Common mistakes

Six matrix bugs (all messages are real)

1. `|m[1][2]|` on an `int[,]`: `error CS0022: Wrong number of indices inside []; expected 2`. Write `|m[1, 2]|`.
2. Rows of different lengths in an initializer, `| 1, 2 , 3 |`: `error CS0847: An array initializer of length '2' is expected`. Every row needs the same length — or use a jagged array.
3. `|for (int r = 0; r < m.Length; r++)|`: `Length` is 12 for 3×4, so `m[3, 0]` throws

`IndexOutOfRangeException`. Use `GetLength(0)` and `GetLength(1)`.

4. Swapped sizes, `|c < m.GetLength(0)|`: correct on a 3×3 matrix, wrong on 3×4. Test with a non-square matrix.
5. Accumulator declared before the outer loop: the second "row sum" includes the first row (`Row 0: 3 Row 1: 10` instead of 7). Reset it inside the outer loop.
6. User numbers vs. indices: forgetting the -1 in `seats[row - 1, seat - 1]` books the wrong seat, and row 5 of 5 crashes. Convert once, right where the array is used; check the range first.

Two more: the transpose created with `|new int[rows, cols]|` instead of `|new int[cols, rows]|`, and a row maximum that starts at 0 instead of at the first element of the row.

>_ 10 Summary

- `|int[,] m = new int[rows, cols];|` creates a rectangular matrix; `|m[r, c]|` is row r, column c, both from 0. An initializer `| 1, 2, 3 , 4, 5, 6 |` gives the shape by its braces.
- `GetLength(0)` = rows, `GetLength(1)` = columns, `Length` = all elements, `Rank` = dimensions.
- Two nested loops visit every element; the outer loop chooses the line (row or column). Print with a width, `{x,5}`, and `WriteLine` after each row.
- Row/column accumulators are reset inside the outer loop; maxima start with the first element.
- Diagonals: `a[i, i]` and `a[i, n - 1 - i]`, one loop. Transpose: `t[c, r] = a[r, c]` with swapped sizes. Symmetric: compare the upper triangle with its mirror.
- $A + B$ and $k \cdot A$ are element by element; $C = A \cdot B$ needs $\text{cols}(A) = \text{rows}(B)$, $C_{ij} = \sum_k A_{ik} B_{kj}$, three nested loops, n^3 multiplications, and $A \cdot B \neq B \cdot A$ in general.
- Grids (grade books, seats, boards, images) are matrices; convert "numbers from 1" to indices once.
- `int[][]` is an array of rows, each with its own length.

>_ 11 Glossary (English – Turkish)

English	Turkish	Meaning
matrix, 2-D array	matris, iki boyutlu dizi	table of elements with two indices
row / column	satır / sütun	horizontal line / vertical line of a matrix
rectangular array	dikdörtgen dizi	`int[,]`: all rows have the same length
jagged array	düzensiz (çentikli) dizi	`int[][]`: array of arrays, rows of different lengths
dimension	boyut	`GetLength(0)` rows, `GetLength(1)` columns
nested loop	iç içe döngü	a loop inside the body of another loop
row-major order	satır öncelikli sıra	visit row 0, then row 1, ...
square matrix	kare matris	as many rows as columns
main diagonal	ana köşegen	elements with $r = c$
anti-diagonal	yan köşegen	elements with $r + c = n - 1$
trace	iz	sum of the main diagonal
identity matrix	birim matris	1 on the main diagonal, 0 elsewhere
transpose	devrik (transpoz)	rows and columns swapped
symmetric matrix	simetrik matris	equal to its transpose
scalar multiplication	skalerle çarpma	every element times the same number
matrix multiplication	matris çarpımı	$C_{ij} = \sum_k A_{ik} B_{kj}$
upper / lower triangle	üst / alt üçgen	elements above ($c > r$) / below ($c < r$) the diagonal
pixel / brightness	piksel / parlaklık	one image point / its grey value

>_ 12 Exercises

12.1 Check yourself: predict the output

P1.

```
int[,] q = { { 1, 2, 3 }, { 4, 5, 6 },
             { 7, 8, 9 } };
int s = 0;
for (int r = 0; r < 3; r++)
{
    for (int c = 0; c < 3; c++)
    {
        if (c >= r)
        {
            s += q[r, c];
        }
    }
}
Console.WriteLine(s);
```

P2.

```
int[,] g = new int[4, 6];
Console.WriteLine(
    $"{g.Length} {g.GetLength(0)}");
```

P3.

```
int[,] m = { { 1, 2, 3 }, { 4, 5, 6 } };
for (int c = 0; c < 3; c++)
{
    for (int r = 0; r < 2; r++)
    {
        Console.Write($"{m[r, c]} ");
    }
    Console.WriteLine();
}
```

P4. What happens to `a`? Fix it.

```
int[,] a = { { 1, 2 }, { 3, 4 } };
for (int r = 0; r < 2; r++)
{
    for (int c = 0; c < 2; c++)
    {
        int temp = a[r, c];
        a[r, c] = a[c, r];
        a[c, r] = temp;
    }
}
```

12.2 In-class exercises (pairs)

A. ⋆ **Max of each row.** Read a 3×4 matrix; print the largest value of every row and its column index.

B. ⋆ $A - B$**.** Subtract two matrices of the same size; print "not defined" if the sizes differ.

C. ⋆⋆ **Is it the identity?** Check whether a square matrix is the identity matrix.

D. ⋆⋆ **Border sum.** Sum of the elements on the border (first/last row, first/last column).

E. ⋆⋆ **Study planner.** A 7×4 matrix holds the study hours for 7 days of the week (rows) in 4 weeks (columns). Print the total per day of the week and per week.

F. ⋆⋆⋆ **Rotate 90°.** Rotate an $n \times n$ matrix clockwise into a new matrix.

12.3 Homework

H1. ⋆⋆ **Validated input.** Extend Example 2: the sizes must be $1 \ldots 10$ and every element an integer; ask again until the input is valid (`TryParse` loops, Week 7).

H2. ⋆⋆⋆ **Saddle point.** An element is a *saddle point* if it is the smallest in its row and the largest in its column. Find all saddle points of $\begin{pmatrix} 3 & 8 & 5 \\ 7 & 9 & 10 \\ 1 & 4 & 6 \end{pmatrix}$.

H3. ⋆⋆ **Cinema, improved.** Change Example 9 so that the seat map is printed after every booking and no more bookings are accepted when the hall is full.

H4. ⋆ **Final grade column.** Add a fourth column "Final grade" to the grade book: $0.3 \cdot \text{quiz} + 0.3 \cdot \text{midterm} + 0.4 \cdot \text{final}$.

H5. ⋆ **By hand.** Compute $\left(\begin{smallmatrix} 2 & 0 \\ 1 & 3 \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} 1 & 4 \\ 2 & 5 \end{smallmatrix}\right)$ and check the result with `11_07` (replace the two initializers).

H6. ⋆ **Repeated code.** Count how many times the "print a matrix" loops were written in today's examples. What would you like to write instead?

>_ 13 References

- Microsoft Learn — Arrays (single-dimensional, multidimensional and jagged arrays): https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/arrays
- Microsoft Learn — `Array.GetLength(Int32)` method: https://learn.microsoft.com/en-us/dotnet/api/system.array.getlength
- Microsoft Learn — `Array.Rank` property: https://learn.microsoft.com/en-us/dotnet/api/system.array.rank
- Microsoft Learn — Iteration statements (`for`, `foreach`, `while`): https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/iteration-statements
- Microsoft Learn — Composite formatting (alignment component `{x,5}`): https://learn.microsoft.com/en-us/dotnet/standard/base-types/composite-formatting
- Microsoft Learn — `IndexOutOfRangeException` class: https://learn.microsoft.com/en-us/dotnet/api/system.indexoutofrangeexception